

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



MÔN PHÁT TRIỂN ỨNG DỤNG IOT

Nhóm 4

ĐỀ TÀI :

**HỆ THỐNG GIÁM SÁT VÀ CẢNH BÁO RÒ RỈ KHÍ GAS
QUA TELEGRAM**

MỤC LỤC

I. Đặt Vấn Đề:	4
II. NỘI DUNG:	4
1. Các nội dung thực hiện:	4
1.1 Bảng phân công công việc:	4
2. Thiết kế mạch IoT	5
2.1 Sơ đồ Logic của hệ thống:	5
2.2 Chức năng của linh kiện:	5
a. Vi điều khiển ESP8266	5
b. Cảm biến khí Gas (Giả lập bằng Biến trở)	6
c. Còi báo động (Buzzer)	6
d. Nền tảng Telegram Bot	6
2.3 Nguyên lý hoạt động:	6
2.4 Mạch vật lý:	7
III. Cài Đặt Hệ Thống:	7
1. Quy trình cài đặt:	7
2. Công cụ sử dụng	8
3. Thuật Toán Hoạt Động:	8
a. Khởi tạo hệ thống	8
b. Giám sát môi trường	8
c. Xử lý báo động và Gửi cảnh báo	9
IV. Kết Quả Thực Hiện :	9
1. Telegram hiển thị kết quả	9
2. Kết quả gửi dữ liệu	10
3. Mô Phỏng wokwi:	10
V. Kết Luận Và Hướng Phát Triển:	11
1. Công việc đã thực hiện:	11
2. Hướng phát triển:	11
VI. Mã nguồn:	12
Phụ lục A. Mã nguồn chương trình ESP32 (main.cpp):	12
Phụ lục B. Mã nguồn cấu hình môi trường (wokwi.toml):	13
Phụ Lục C. Mã nguồn sơ đồ mạch Diagram.json:	14
Phụ lục D. Tài liệu tham khảo:	14

I. Đặt Vấn Đề:

Trong đời sống hiện đại, bếp gas và các hệ thống cung cấp khí gas được sử dụng vô cùng phổ biến tại các hộ gia đình, nhà hàng và cơ sở kinh doanh. Tuy nhiên, tình trạng rò rỉ khí gas lại là một trong những nguyên nhân hàng đầu gây ra các vụ cháy nổ thương tâm, để lại hậu quả nặng nề về người và tài sản.

Hầu hết các sự cố xảy ra do người dùng không phát hiện kịp thời sự rò rỉ khi đang ngủ hoặc không có mặt tại hiện trường. Xuất phát từ thực tế đó, nhóm chúng em quyết định thực hiện đề tài "Hệ thống giám sát và cảnh báo rò rỉ khí gas qua Telegram".

Hệ thống sử dụng vi điều khiển ESP32 kết hợp cùng cảm biến khí gas để theo dõi nồng độ khí 24/7. Điểm nổi bật của hệ thống là khả năng phát tín hiệu báo động tại chỗ bằng còi hú, đồng thời tự động gọi API để gửi tin nhắn khẩn cấp đến điện thoại của người dùng thông qua ứng dụng Telegram ngay khi nồng độ gas vượt mức an toàn. Giải pháp này giúp người dùng có biện pháp xử lý kịp thời, ngăn chặn thảm họa dù ở bất cứ đâu.

II. NỘI DUNG:

1. Các nội dung thực hiện:

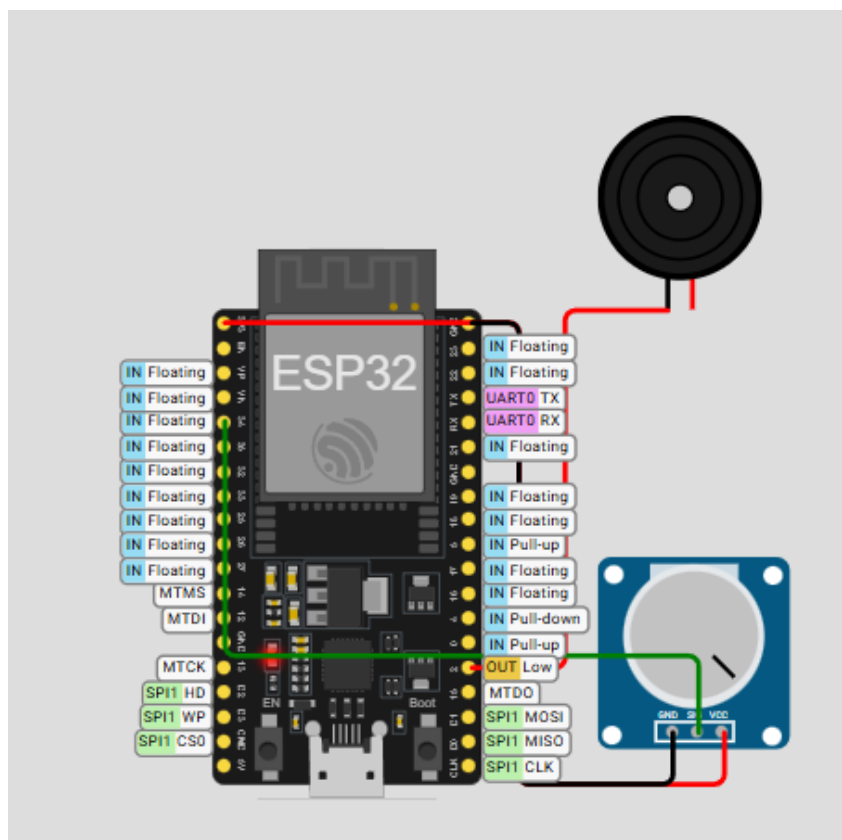
1.1 Bảng phân công công việc:

STT	Nội dung công việc	Tiến độ
1	Thiết kế sơ đồ mạch IoT và thiết lập mô phỏng trên nền tảng Wokwi.	100%
2	Cấu hình Telegram Bot thông qua BotFather, lấy API Token và Chat ID.	100%
3	Lập trình vi điều khiển ESP32: Kết nối WiFi ảo, thiết lập giao tiếp HTTPS với API Telegram.	100%
4	Lập trình xử lý logic: Đọc dữ liệu biến trở giả lập cảm biến Gas, cài đặt ngưỡng cảnh báo và điều khiển còi hú.	100%
5	Kiểm thử hệ thống, đánh giá độ trễ gửi tin nhắn và tổng hợp viết báo cáo tiểu luận.	100%

Bảng 1: Bảng liệt kê các công việc thực hiện trong đề tài

2. Thiết kế mạch IoT

2.1 Sơ đồ Logic của hệ thống:



2.2 Chức năng của linh kiện:

a. Vi điều khiển ESP8266

Đóng vai trò là bộ não trung tâm của hệ thống. ESP32 chịu trách nhiệm đọc dữ liệu tín hiệu Analog từ cảm biến (hoặc biến trở giả lập), xử lý so sánh với ngưỡng an toàn được cài đặt sẵn. Nhờ được tích hợp sẵn module Wi-Fi, ESP32 có khả năng kết nối Internet để gọi API và gửi tin nhắn cảnh báo qua nền tảng Telegram.

b. Cảm biến khí Gas (Giả lập bằng Biến trở - Potentiometer)

Trong môi trường mô phỏng Wokwi không có sẵn cảm biến Gas thực tế (như MQ-6, MQ-2), hệ thống sử dụng một Biến trở để thay thế. Biến trở này thay đổi mức điện áp đầu ra (tạo ra giá trị từ 0 đến 4095), giúp giả lập sự biến thiên của nồng độ khí gas trong môi trường từ mức an toàn đến mức nguy hiểm.

c. Còi báo động (Buzzer):

Hoạt động như một thiết bị cảnh báo tại chỗ. Khi vi điều khiển phát hiện nồng độ gas vượt ngưỡng rò rỉ, nó sẽ xuất tín hiệu điện áp (HIGH) làm còi phát ra âm thanh lớn, báo động cho những người đang có mặt ở khu vực xung quanh biết để sơ tán và xử lý.

d. Nền tảng Telegram Bot

Hoạt động như một công cụ truyền thông tin từ xa. Thông qua Telegram Bot API, hệ thống có thể gửi ngay lập tức các tin nhắn văn bản cảnh báo rò rỉ gas trực tiếp vào điện thoại của người dùng một cách nhanh chóng, miễn phí và bảo mật.

2.3 Nguyên lý hoạt động:

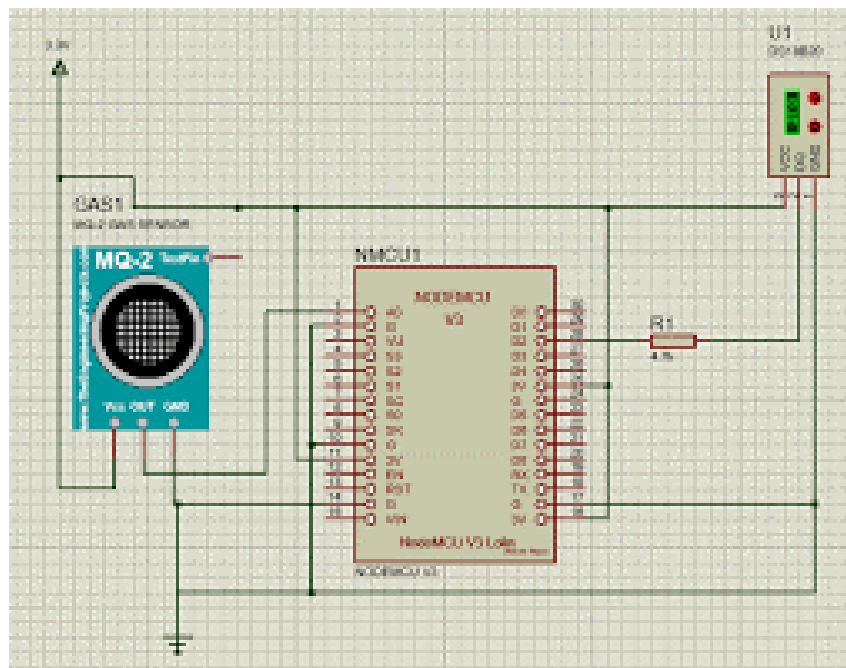
Hệ thống hoạt động theo một chu trình giám sát và cảnh báo liên tục như sau:

Bước 1 (Thu thập dữ liệu): Sau khi khởi động và kết nối thành công với mạng Wi-Fi ảo, vi điều khiển ESP32 sẽ gửi một tin nhắn thông báo hệ thống đã sẵn sàng qua Telegram. Sau đó, nó đi vào vòng lặp liên tục đọc giá trị Analog từ cảm biến khí gas (giả lập bằng biến trở) với tần suất 1 giây/lần.

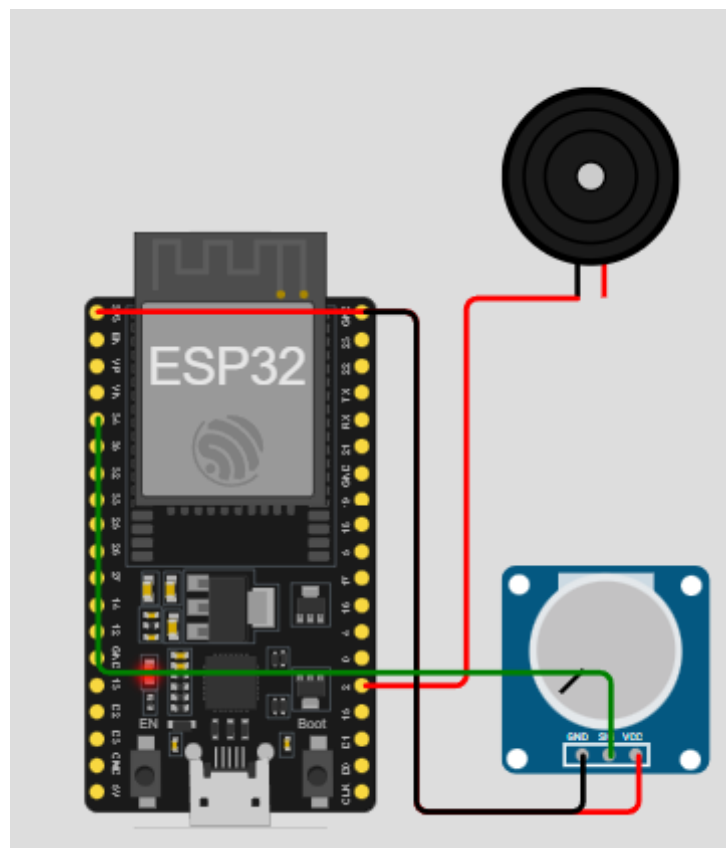
Bước 2 (Xử lý và cảnh báo tại chỗ): ESP32 tiến hành so sánh giá trị đọc được với một ngưỡng rò rỉ gas đã được lập trình sẵn (ngưỡng 2000). Nếu nồng độ gas nằm trong mức an toàn (< 2000), hệ thống giữ còi báo động ở trạng thái tắt. Nếu nồng độ gas tăng cao vượt mức ngưỡng (≥ 2000), ESP32 lập tức xuất tín hiệu điều khiển bật còi báo động (Buzzer) để phát âm thanh cảnh báo tại hiện trường.

Bước 3 (Cảnh báo từ xa qua Internet): Cùng lúc với việc bật còi, ESP32 sẽ kết nối với API của Telegram thông

qua thư viện UniversalTelegramBot. Nó đóng gói một tin nhắn chứa nội dung cảnh báo khẩn cấp kèm theo giá trị nồng độ gas đo được hiện tại và gửi trực tiếp đến Chat ID của người dùng. Để hệ thống hoạt động ổn định, tránh việc Telegram đánh dấu là "spam" do gửi quá nhiều tin nhắn liên tục, ESP32 được lập trình bộ đếm thời gian (millis) để chỉ gửi tối đa một tin nhắn cảnh báo sau mỗi 10 giây nếu khí gas vẫn tiếp tục rò rỉ.



2.4 Mạch vật lý:



III. Cài Đặt Hệ Thống:

1. Quy trình cài đặt:

Quy trình triển khai hệ thống IoT được thực hiện theo các bước sau:

Bước 1: Chuẩn bị môi trường phát triển gồm VS Code, PlatformIO, tiện ích mở rộng Wokwi và thiết lập Bot trên ứng dụng Telegram.

Bước 2: Thiết kế và mô phỏng mạch trên Wokwi với các linh kiện: ESP32 DevKit V1, Biến trở (giả lập cảm biến Gas), Còi báo động (Buzzer).

Bước 3: Lập trình ESP32 bằng ngôn ngữ C++ để:

- Kết nối mạng WiFi (Wokwi-GUEST).
- Khởi tạo kết nối HTTPS bảo mật với Telegram Bot API.
- Đọc dữ liệu Analog từ biến trở giả lập nồng độ Gas.
- Xử lý logic so sánh giá trị với ngưỡng báo động.
- Điều khiển còi hú và gửi tin nhắn cảnh báo về điện thoại.

Bước 4: Cấu hình Telegram Bot qua BotFather để lấy API Token và tra cứu Chat ID của người dùng.

Bước 5: Tích hợp trình mô phỏng Wokwi vào VS Code thông qua file cấu hình diagram.json và wokwi.toml.

Bước 6: Kiểm thử hệ thống: Xoay biến trở để thay đổi nồng độ gas và đánh giá phản xạ của còi báo động cũng như tốc độ nhận tin nhắn trên Telegram.

2. Công cụ sử dụng

Arduino Framework: Khung lập trình tiêu chuẩn dùng để viết mã nguồn cho vi điều khiển ESP32.

Visual Studio Code: Trình soạn thảo mã nguồn chuyên nghiệp, nơi quản lý toàn bộ dự án.

PlatformIO: Tiện ích hỗ trợ quản lý thư viện (UniversalTelegramBot, ArduinoJson), biên dịch mã nguồn và nạp chương trình.

Wokwi Simulator: Nền tảng mô phỏng mạch điện tử và kết nối WiFi ảo, giúp kiểm tra hệ thống mà không cần phần cứng thật.

Telegram Bot API: Cổng giao tiếp trung gian, cho phép ESP32 gửi thông tin cảnh báo trực tiếp đến ứng dụng Telegram của người dùng.

Thư viện UniversalTelegramBot: Thư viện mã nguồn mở giúp đơn giản hóa việc gửi/nhận tin nhắn qua Telegram API.

Thư viện ArduinoJson: Dùng để đóng gói và xử lý các chuỗi dữ liệu JSON khi giao tiếp với máy chủ Telegram.

3. Thuật Toán Hoạt Động:

a. Khởi tạo hệ thống

- Khởi tạo Serial Monitor để theo dõi quá trình debug hệ thống.
- Cấu hình chân GPIO 34 làm đầu vào Analog (đọc biến trở) và chân GPIO 2 làm đầu ra Digital (điều khiển còi).
- Thiết lập kết nối WiFi với điểm truy cập Wokwi-GUEST.
- Khởi tạo đối tượng Telegram Bot với Token đã cấp.
- Gửi tin nhắn chào mừng "Hệ thống giám sát Gas đã sẵn sàng" qua Telegram để xác nhận kết nối thành công.

b. Giám sát môi trường

- Hệ thống hoạt động theo chu kỳ quét liên tục (mỗi 1 giây):

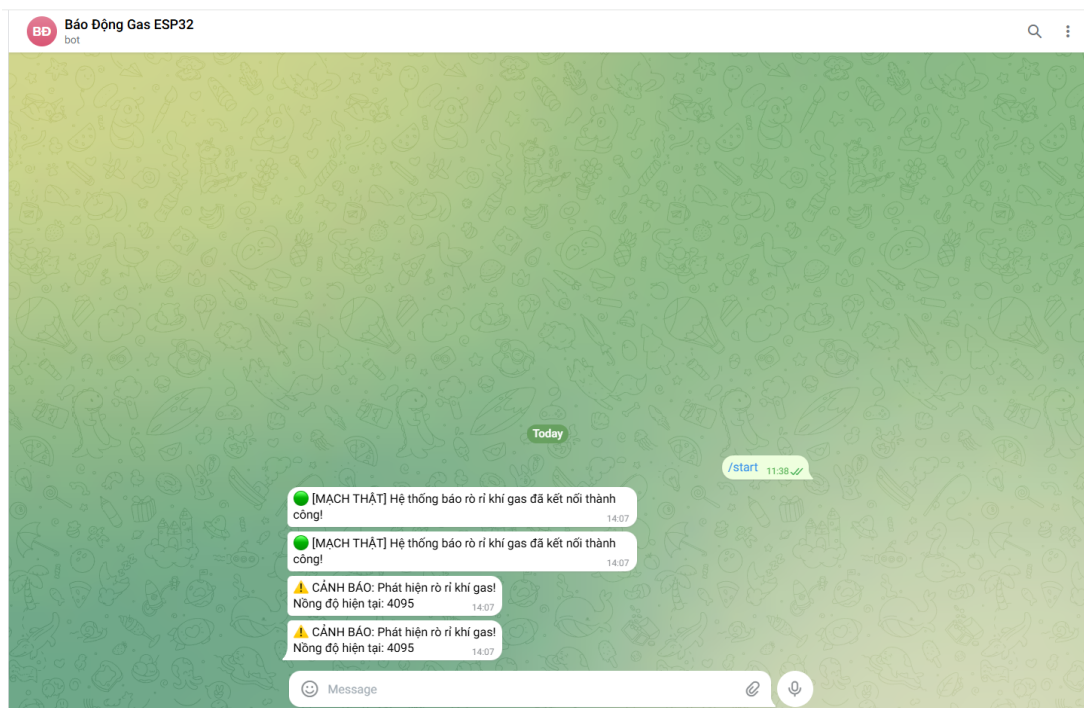
- ESP32 đọc giá trị điện áp từ chân Analog 34 (giá trị từ 0 đến 4095).
- Chuyển đổi giá trị đọc được thành thông tin nồng độ gas mô phỏng.
- Hiển thị giá trị đo được lên màn hình Serial để người dùng theo dõi trạng thái hiện tại của môi trường.

c. Xử lý báo động và Gửi cảnh báo

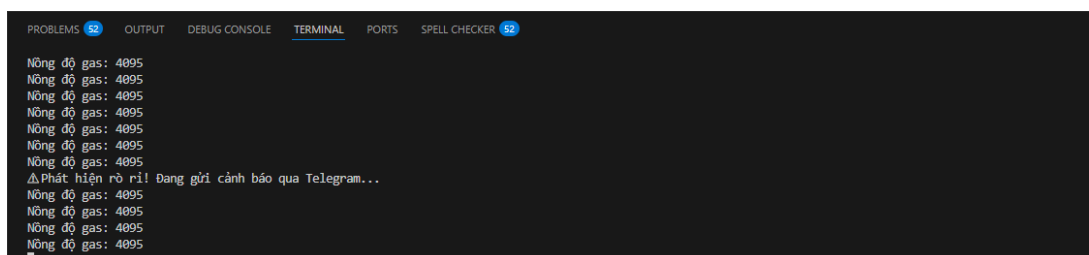
- Khi giá trị cảm biến vượt quá ngưỡng cài đặt (ví dụ: > 2000):
Cảnh báo tại chỗ: ESP32 xuất tín hiệu mức cao (HIGH) ra chân D2 để kích hoạt còi báo động (Buzzer) kêu liên tục.
Cảnh báo từ xa: Hệ thống gọi hàm gửi tin nhắn của Telegram Bot để gửi thông báo khẩn cấp: "CẢNH BÁO: Phát hiện rò rỉ khí Gas! Giá trị đo được: [Giá trị]".
Cơ chế chống lặp: Sử dụng biến thời gian để giới hạn việc gửi tin nhắn (ví dụ: chỉ gửi 1 tin mỗi 10 giây) nhằm tránh gây phiền hà cho người dùng và tránh bị Telegram chặn (Spam).
- Khi giá trị cảm biến trở về mức an toàn (< 2000): Xuất tín hiệu mức thấp (LOW) để tắt còi báo động và đưa hệ thống về trạng thái giám sát bình thường.

IV. Kết Quả Thực Hiện :

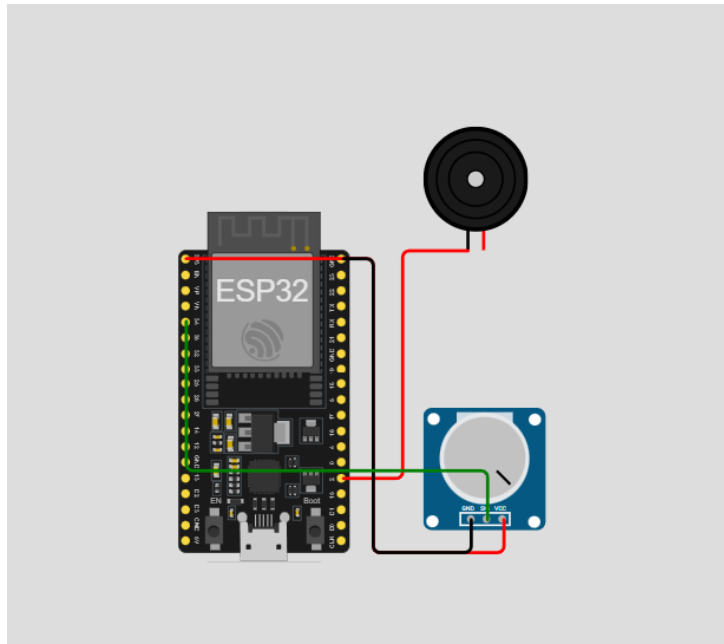
1. Telegram hiển thị kết quả



2. Kết quả gửi dữ liệu



3. Mô Phỏng wokwi:



V. Kết Luận Và Hướng Phát Triển:

1. Công việc đã thực hiện:

Trong quá trình thực hiện đề tài, nhóm đã hoàn thành các nội dung chính sau:

- Xây dựng thành công hệ thống IoT giám sát nồng độ khí gas và cảnh báo rò rỉ qua ứng dụng Telegram.
- Thiết kế và mô phỏng mạch điện tử trên Wokwi gồm: vi điều khiển ESP32, biến trở (giả lập cảm biến Gas) và còi báo động (Buzzer).
- Lập trình thành công vi điều khiển để:
 - Kết nối mạng WiFi ảo của Wokwi.
 - Đọc và xử lý dữ liệu Analog từ cảm biến.
 - Thiết lập giao thức HTTPS để giao tiếp với Telegram Bot API.
 - Tự động gửi tin nhắn cảnh báo thời gian thực khi phát hiện rò rỉ.
- Xây dựng logic điều khiển còi hú báo động tại chỗ song song với cảnh báo từ xa.
- Kiểm thử hệ thống hoạt động ổn định, tin nhắn gửi về điện thoại có độ trễ thấp.

2. Hướng phát triển:

Trong tương lai, đề tài có thể được phát triển theo các hướng sau:

- Triển khai hệ thống trên phần cứng thực tế sử dụng cảm biến khói/gas chuyên dụng MQ-2 hoặc MQ-6.
- Mở rộng thêm các tính năng an toàn như:
 - Tự động ngắt van gas bằng động cơ Servo hoặc Relay.
 - Tích hợp quạt thông gió tự động khi phát hiện nồng độ gas cao.
- Phát triển giao diện giám sát trực quan hơn trên Web hoặc Mobile App để theo dõi lịch sử nồng độ gas.
- Tích hợp thêm module SIM để gọi điện hoặc nhắn tin SMS trực tiếp trong trường hợp mất kết nối WiFi.
- Nâng cao bảo mật và xác thực người dùng để tránh việc điều khiển nhầm lẫn từ bên ngoài.

VI. Mã nguồn:

Phụ lục A. Mã nguồn chương trình ESP32 (main.cpp):

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>

// --- THÔNG TIN WIFI ---
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// --- THÔNG TIN TELEGRAM BOT ---
#define BOT_TOKEN "8777369752:AAgUvBPL-5pvq1FPM7ZzQAxNWmGNJcDGEP4"
#define CHAT_ID "8796555849"

WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);

const int gasPin = 34;
const int buzzerPin = 2;
const int gasThreshold = 2000;
unsigned long lastTimeBotRan = 0;
const unsigned long botRequestDelay = 10000;

void setup() {
    Serial.begin(115200);
    pinMode(gasPin, INPUT);
    pinMode(buzzerPin, OUTPUT);
    digitalWrite(buzzerPin, LOW);

    Serial.print("Dang ket noi WiFi...");
    WiFi.begin(ssid, password);
    client.setInsecure();

    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }

    Serial.println("\nWiFi da ket noi thanh cong!");
    bot.sendMessage(CHAT_ID, " He thong bao ro ri khi gas da san sang!", "");
}

void loop() {
    int gasLevel = analogRead(gasPin);
    Serial.print("Nong do gas: ");
    Serial.println(gasLevel);

    if (gasLevel > gasThreshold) {
        digitalWrite(buzzerPin, HIGH);
        if (millis() - lastTimeBotRan > botRequestDelay) {
```

```

Serial.println(" Phat hien ro ri! Dang gui Telegram...");
String message = " CANH BAO: Phat hien ro ri khi gas!\nNong do: " + String
    (gasLevel);
bot.sendMessage(CHAT_ID, message, "");
lastTimeBotRan = millis();
}
} else {
    digitalWrite(buzzerPin, LOW);
}
delay(1000);
}

```

Phụ lục B. Mã nguồn cấu hình môi trường (wokwi.toml):

```

[wokwi]
version = 1
firmware = ".pio/build/esp32doit-devkit-v1/firmware.bin"
elf = ".pio/build/esp32doit-devkit-v1/firmware.elf"

```

Phụ Lục C. Mã nguồn sơ đồ mạch Diagram.json:

```

{
  "version": 1,
  "author": " N g u y e n      i      Q u      ",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": -150, "left": -100, "attrs": {} },
    { "type": "wokwi-buzzer", "id": "buzzer1", "top": -150, "left": 100, "attrs": {} }
  ],
  "connections": [
    [ "esp:GND.1", "pot1:GND", "black", [ "v0" ] ],
    [ "esp:3V3", "pot1:VCC", "red", [ "v0" ] ],
    [ "esp:34", "pot1:SIG", "green", [ "v0" ] ],
    [ "esp:GND.2", "buzzer1:GND", "black", [ "v0" ] ],
    [ "esp:2", "buzzer1:SIG", "red", [ "v0" ] ]
  ]
}

```

Phụ lục D. Tài liệu tham khảo:

Tài liệu giảng dạy môn “Phát triển ứng dụng IoT” – Võ Việt Dũng.

Trang tài liệu chính thức của thư viện UniversalTelegramBot.

Wokwi Documentation cho mô phỏng ESP32 và WiFi.